

FATEC — Desenvolvimento de Software Multiplataforma
Computação em Nuvem II

Relatório Técnico

Armazenamento em Nuvem com AWS

Atividade 3 — Amazon S3 · Amazon RDS · Amazon DynamoDB

Grupo	Skyfall
Integrantes	Eduardo da Silva Fontes Eduardo Kuwahara Jr. Fábio Hiroshi João Vitor Rossi Ferreira Kathellyn Caroline Alves dos Santos Paulo Henrique Martins de Almeida Victor Daniel
Projeto (API)	Sistema Tutor de Aprendizagem
Turma / Semestre	DSM — 6º semestre / 2026
Data de entrega	01/10/2026
Conta AWS utilizada	161800939960
Região	us-east-1 (N. Virginia)
Link do bucket S3 (site)	http://fatec-dsm-skyfall.s3-website-us-east-1.amazonaws.com
Link do repositório	https://github.com/Equipe-Skyfall/Computacao-em-nuvem-atv3.git
Versão	Versão 1 (console + AWS CLI) e Versão 2 (uso via API / SDK)

Contexto e decisão de armazenamento

O projeto de API do grupo é um **Sistema Tutor de Aprendizagem**: alunos acessam trilhas de estudo, assistem aulas, respondem exercícios e conversam com um tutor que recomenda conteúdos. Esse sistema produz três tipos de dado com perfis de acesso muito diferentes, e por isso o grupo decidiu usar um serviço de armazenamento para cada um, em vez de forçar tudo em um único banco.

Dado do Sistema Tutor	Serviço AWS	Justificativa
Materiais de aula (PDF, imagens, vídeos), site estático	Amazon S3	Objetos binários grandes, leitura direta por URL, custo por GB muito baixo, versionamento e ciclo de vida nativos.
Alunos, cursos, matrículas e notas	Amazon RDS (PostgreSQL)	Dados relacionais com integridade (chaves estrangeiras, UNIQUE, CHECK), JOINS e relatórios agregados.
Log de eventos de acesso em tempo real	Amazon DynamoDB	Altíssimo volume de escrita, acesso sempre por chave (aluno ou curso + tempo), escala horizontal automática.

Todos os recursos foram criados por scripts de AWS CLI versionados no repositório (v1-cli/), o que permite reproduzir o ambiente e registrar as saídas como evidência (evidencias/*.log). Os nomes seguem o padrão `fatec-skyfall-<recurso>` e todos receberam as tags `Projeto=API-Tutor` e `Grupo=skyfall`.

Seção 1 — Amazon S3

1.1 Configuração do Bucket

Nome	fatec-dsm-skyfall
Região	us-east-1 — US East (N. Virginia)
Object Ownership	Bucket owner enforced (ACLs desabilitadas — padrão atual da AWS)
Block Public Access	ACLs públicas bloqueadas; políticas públicas permitidas (para liberar só o prefixo publico/)
Versionamento	Habilitado
Criptografia	SSE-S3 (padrão, AES-256)
Tags	Projeto=API-Tutor, Disciplina=Nuvem-II, Grupo=skyfall

Por que us-east-1? É a região mais barata da AWS e a primeira a receber novos serviços; também é a região liberada no AWS Academy Learner Lab. A alternativa natural para usuários brasileiros seria sa-east-1 (São Paulo), que tem latência menor (cerca de 10–20 ms contra 110–130 ms a partir de SP), porém custo significativamente maior (S3 Standard ≈ US\$ 0,0405/GB-mês em São Paulo contra US\$ 0,023/GB-mês na Virgínia). Para um projeto acadêmico, o custo pesou mais; em produção, os materiais seriam servidos via CloudFront, o que elimina a diferença de latência para o usuário final.

Comando utilizado (trecho de `v1-cli/01_s3.sh`):

```
aws s3api create-bucket --bucket fatec-dsm-skyfall --region us-east-1
aws s3api put-bucket-versioning --bucket fatec-dsm-skyfall \
  --versioning-configuration Status=Enabled
aws s3api put-public-access-block --bucket fatec-dsm-skyfall \
```

```
--public-access-block-configuration \
BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=false,RestrictPublicBuckets=false

atv3 $ aws s3api list-buckets --query "Buckets[?Name=='$BUCKET']" --output table
```

ListBuckets		
BucketArn	CreationDate	Name
arn:aws:s3:::fatec-dsm-skyfall	2026-10-01T22:50:18+00:00	fatec-dsm-skyfall

1.2 Arquivos Enviados e Permissões

Arquivo (chave)	Tipo	Acesso	Critério
publico/logo-tutor.png	Imagem PNG	Público	Identidade visual exibida no site e no app.
publico/guia-do-aluno.pdf	PDF	Público	Material didático aberto a qualquer visitante.
publico/notas-da-versao.txt	TXT	Público	Changelog do sistema; usado na demonstração de versionamento.
privado/alunos.csv	CSV	Privado	Contém dados pessoais (nome, RA, e-mail) — protegidos pela LGPD.
privado/config-trilhas.json	JSON	Privado	Configuração interna das trilhas e chave de gabarito; não pode vaziar.
index.html / error.html	HTML	Público	Páginas do site estático (item opcional).

Critério de acesso: é público apenas o que é material de divulgação ou didático; tudo que contém dado pessoal ou informação interna é privado (princípio do menor privilégio). Em vez de configurar permissão objeto por objeto (ACLs, hoje desabilitadas por padrão), o grupo usou uma **bucket policy por prefixo**: qualquer arquivo colocado em `publico/` fica legível pela internet e qualquer arquivo em `privado/` só é acessível com credenciais da conta ou por URL pré-assinada temporária (usada na Versão 2). Essa abordagem é auditável em um único documento e escala para milhares de arquivos.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "LeituraPublicaMateriais",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": [
      "arn:aws:s3:::fatec-dsm-skyfall/publico/*",
      "arn:aws:s3:::fatec-dsm-skyfall/index.html",
      "arn:aws:s3:::fatec-dsm-skyfall/error.html"
    ]
  }]
}
```

Teste de acesso anônimo realizado pelo script (saída em `evidencias/s3.log`):

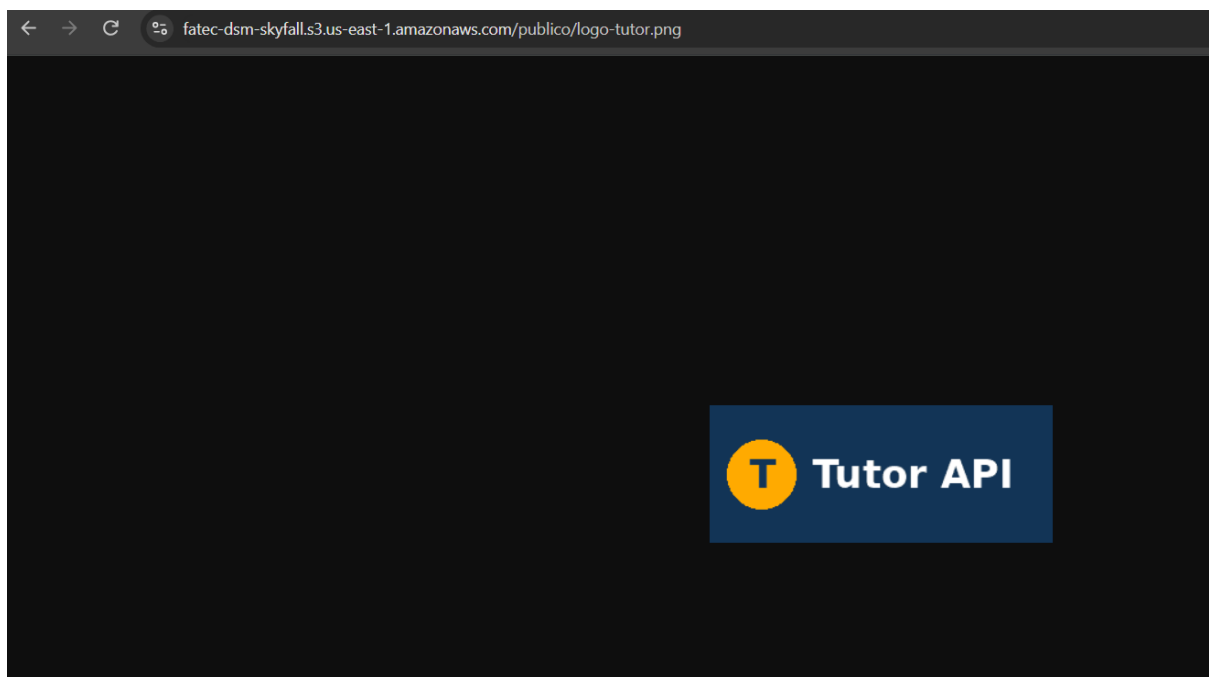
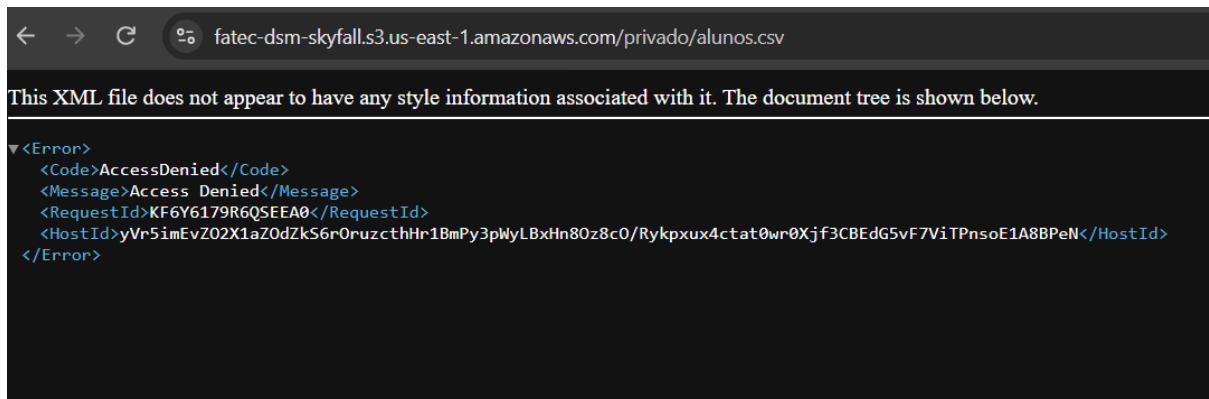
```
PUBLICO https://fatec-dsm-skyfall.s3.us-east-1.amazonaws.com/publico/logo-tutor.png -> HTTP
200
PRIVADO https://fatec-dsm-skyfall.s3.us-east-1.amazonaws.com/privado/alunos.csv -> HTTP
403
```

```

atv3 $ aws s3 ls s3://$BUCKET --recursive --human-readable
2026-10-01 22:51:15 106 Bytes error.html
2026-10-01 22:51:15 590 Bytes index.html
2026-10-01 22:51:14 295 Bytes privado/alunos.csv
2026-10-01 22:51:14 231 Bytes privado/config-trilhas.json
2026-10-01 22:51:13 687 Bytes publico/guia-do-aluno.pdf
2026-10-01 22:51:13 990 Bytes publico/logo-tutor.png
2026-10-01 22:54:26 181 Bytes publico/notas-da-versao.txt

```

URL pública de teste: <https://fatec-dsm-skyfall.s3.us-east-1.amazonaws.com/publico/logo-tutor.png>



1.3 Versionamento

O versionamento foi habilitado logo após a criação do bucket (put-bucket-versioning Status=Enabled). Com ele ativo, cada upload de um arquivo com a mesma chave cria uma nova versão com um **VersionId** próprio em vez de sobrescrever o anterior; exclusões viram apenas um **delete marker**, permitindo recuperação. Para demonstrar, o arquivo `publico/notas-da-versao.txt` foi enviado duas vezes: a versão 1.0 (lista inicial de funcionalidades) e a versão 2.0 (inclui o registro de eventos em DynamoDB e os materiais em S3).

```

aws s3 cp s3-arquivos/versao2/publico/notas-da-versao.txt \
s3://fatec-dsm-skyfall/publico/notas-da-versao.txt

```

```
aws s3api list-object-versions --bucket fatec-dsm-skyfall \
  --prefix publico/notas-da-versao.txt --output table
```

VersionId	IsLatest	LastModified	Conteúdo
tZNZQm4ZdT0FtguBtMEM_2OJ3IsV4PUD	True	2026-10-01 22:54:26 UTC	Versão 2.0
7Kq1IIC_3lU.nG_7_yb3SUuEjPEo13rm	False	2026-10-01 22:51:13 UTC	Versão 1.0

Para restaurar a versão anterior basta copiá-la sobre a atual: `aws s3api copy-object --copy-source fatec-dsm-skyfall/publico/notas-da-versao.txt?versionId=<ID-v1>`

```
aws $ aws s3api list-object-versions --bucket $bucket --prefix publico/notas-da-versao.txt \
> --query "Versions[].{VersionId:VersionId,IsLatest:IsLatest,LastModified:LastModified}" --output table
-----
|                               ListObjectVersions                               |
|-----|-----|-----|
| IsLatest | LastModified | VersionId |
|-----|-----|-----|
| True     | 2026-10-01T22:54:26+00:00 | tZNZQm4ZdT0FtguBtMEM_2OJ3IsV4PUD |
| False    | 2026-10-01T22:51:13+00:00 | 7Kq1IIC_3lU.nG_7_yb3SUuEjPEo13rm |
|-----|-----|-----|
```

1.4 Lifecycle Rule

Foram criadas duas regras de ciclo de vida, pensando em custo:

Regra (ID)	Filtro	Ação
privado-para-IA-e-Glacier	Prefixo privado/	Após 30 dias → STANDARD_IA (acesso infrequente); após 90 dias → GLACIER (Flexible Retrieval)
expirar-versoes-antigas	Bucket inteiro	Versões não-atuais são apagadas 30 dias após serem substituídas; uploads multipart incompletos abortados após 7 dias

Justificativa: exportações de alunos e configurações antigas quase não são lidas depois do primeiro mês, então migram para classes mais baratas (o Standard-IA custa cerca de metade do Standard, e o Glacier menos de um quinto). A segunda regra impede que o versionamento faça o custo crescer indefinidamente, mantendo uma janela de 30 dias para recuperar um arquivo sobrescrito por engano.

```

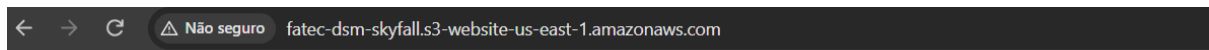
atv3 $ aws s3api get-bucket-lifecycle-configuration --bucket $BUCKET
{
  "TransitionDefaultMinimumObjectSize": "all_storage_classes_128K",
  "Rules": [
    {
      "ID": "privado-para-IA-e-Glacier",
      "Filter": {
        "Prefix": "privado/"
      },
      "Status": "Enabled",
      "Transitions": [
        {
          "Days": 30,
          "StorageClass": "STANDARD_IA"
        },
        {
          "Days": 90,
          "StorageClass": "GLACIER"
        }
      ]
    },
    {
      "ID": "expirar-versoes-antigas",
      "Filter": {
        "Prefix": ""
      },
      "Status": "Enabled",
      "NoncurrentVersionExpiration": {
        "NoncurrentDays": 30
      },
      "AbortIncompleteMultipartUpload": {
        "DaysAfterInitiation": 7
      }
    }
  ]
}

```

1.5 Site estático (opcional)

O bucket também hospeda uma página simples (index.html, com error.html para erros), que exibe o logo e links para os materiais públicos — e um link de teste para o CSV privado, que corretamente retorna erro de acesso. Comando: `aws s3 website s3://fatec-dsm-skyfall/ --index-document index.html --error-document error.html`.

Endpoint: `http://fatec-dsm-skyfall.s3-website-us-east-1.amazonaws.com`



Sistema Tutor de Aprendizagem

Site estatico no Amazon S3 - Grupo Skyfall - FATEC DSM



- [Guia do aluno \(PDF\)](#)
- [Notas da versao \(TXT\)](#)
- [Arquivo privado \(deve dar erro 403\)](#)

Seção 2 — Amazon RDS

2.1 Configuração da Instância

Identificador	fatec-skyfall-rds
Engine / versão	PostgreSQL 18.3 (versão padrão da AWS no momento da criação)
Classe da instância	db.t3.micro — 2 vCPU, 1 GiB RAM (elegível ao free tier)
Multi-AZ	Não (Single-AZ) — fora do free tier e desnecessário para ambiente acadêmico
Armazenamento	20 GiB, General Purpose SSD (gp2)
Acesso público	Sim, mas restrito por Security Group a um único IP (/32)
Banco inicial / usuário	tutordb / tutoradmin
Backup automático	Retenção de 1 dia

```
aws rds create-db-instance --db-instance-identifier fatec-skyfall-rds \
--engine postgres --db-instance-class db.t3.micro \
--allocated-storage 20 --storage-type gp2 --no-multi-az \
--publicly-accessible --vpc-security-group-ids <sg-id> \
--master-username tutoradmin --master-user-password ***** \
--db-name tutordb --backup-retention-period 1
aws rds wait db-instance-available --db-instance-identifier fatec-skyfall-rds
```

Escolhemos PostgreSQL por ser open source, ter tipos mais ricos (NUMERIC, JSONB, arrays) e ser o banco já usado em outras disciplinas do curso. Multi-AZ dobraria o custo; ele é a escolha certa para produção (failover automático em outra zona de disponibilidade), não para este laboratório.

```
> aws query --dbinstances[0].{Status,DBInstanceStatus,Version,EngineVersion,Endpoint,Endpoint.Address} --db
----- DescribeDBInstances -----
+-----+-----+-----+-----+
| Endpoint | Status | Versao |
+-----+-----+-----+-----+
| fatec-skyfall-rds.czgjtc86j0x.us-east-1.rds.amazonaws.com | available | 18.3 |
+-----+-----+-----+-----+
atv3 $
```

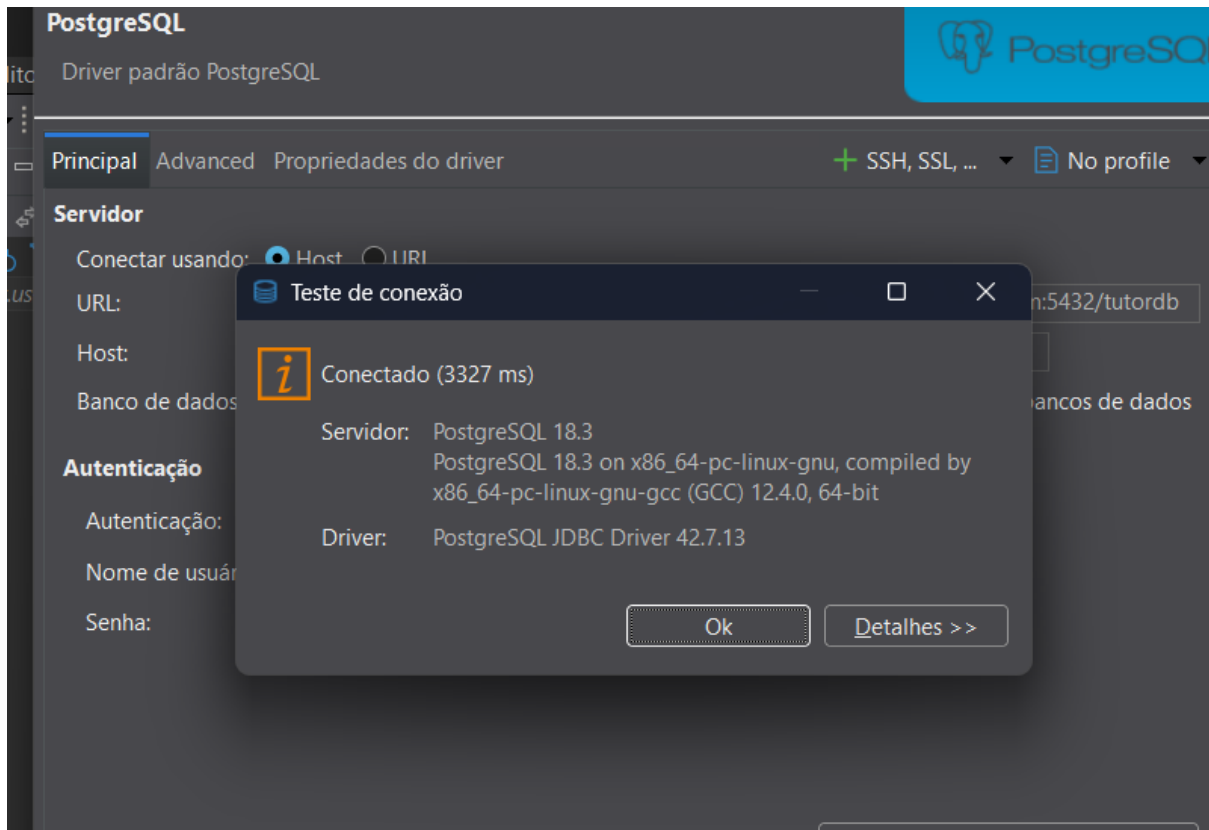
2.2 Conectividade

O script criou o Security Group fatec-skyfall-rds-sg na VPC padrão, com **uma única regra de entrada**: TCP 5432 (PostgreSQL) a partir do IP público do integrante que fez a conexão, no formato <IP>/32 (descoberto automaticamente via <https://checkip.amazonaws.com>). Qualquer outro endereço da internet é recusado, mesmo com a senha correta. Não há regra 0.0.0.0/0.

```
aws ec2 authorize-security-group-ingress --group-id <sg-id> \
--ip-permissions "IpProtocol=tcp,FromPort=5432,ToPort=5432,IpRanges=[{CidrIp=<IP>/32}]"
```

A conexão foi feita com o **DBeaver** (Nova conexão → PostgreSQL), informando o endpoint do RDS, porta 5432, banco tutordb e usuário tutoradmin, com SSL habilitado. IP liberado: **191.201.122.173/32** — conexão feita por **Eduardo Kuwahara**.

```
> --query "SecurityGroupRules[?IsEgress==\`fa
-----
| DescribeSecurityGroupRules
+-----+-----+-----+
| Descricao | Origem | Porta |
+-----+-----+-----+
| None      | 191.201.122.173/32 | 5432 |
+-----+-----+-----+
```



2.3 Dados e Queries

Modelo relacional do Sistema Tutor: curso (4 registros), aluno (12 registros) e matricula (20 registros), que é a tabela associativa N:N com progresso e nota. Os scripts completos estão em `sql/01_schema.sql` e `sql/02_seed.sql`.

```
CREATE TABLE curso (
  id SERIAL PRIMARY KEY,
  codigo VARCHAR(10) NOT NULL UNIQUE,
  nome VARCHAR(100) NOT NULL,
  carga_horaria INTEGER NOT NULL CHECK (carga_horaria > 0)
);
CREATE TABLE aluno (
  id SERIAL PRIMARY KEY,
  ra VARCHAR(13) NOT NULL UNIQUE,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE,
  semestre SMALLINT NOT NULL CHECK (semestre BETWEEN 1 AND 6),
  criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE matricula (
  id SERIAL PRIMARY KEY,
  aluno_id INTEGER NOT NULL REFERENCES aluno(id) ON DELETE CASCADE,
  curso_id INTEGER NOT NULL REFERENCES curso(id),
  progresso NUMERIC(5,2) NOT NULL DEFAULT 0 CHECK (progresso BETWEEN 0 AND 100),
```

```

    nota        NUMERIC(4,2)        CHECK (nota BETWEEN 0 AND 10),
    UNIQUE (aluno_id, curso_id)
);
CREATE INDEX idx_matricula_curso ON matricula(curso_id);

```

Query 1 — SELECT com filtro: alunos do 4º semestre

```
SELECT ra, nome, email FROM aluno WHERE semestre = 4 ORDER BY nome;
```

ra	nome	email
2025001	Ana Souza	ana.souza@fatec.sp.gov.br
2025002	Bruno Lima	bruno.lima@fatec.sp.gov.br
2025006	Felipe Nunes	felipe.nunes@fatec.sp.gov.br
2025010	João Pereira	joao.pereira@fatec.sp.gov.br

Query 2 — JOIN: alunos que concluíram um curso (progresso 100%), por nota

```

SELECT a.nome AS aluno, c.nome AS curso, m.nota
FROM matricula m
JOIN aluno a ON a.id = m.aluno_id
JOIN curso c ON c.id = m.curso_id
WHERE m.progresso = 100
ORDER BY m.nota DESC;

```

aluno	curso	nota
Elisa Prado	Banco de Dados	10.00
Ana Souza	Banco de Dados	9.50
João Pereira	Computação em Nuvem	8.80
Felipe Nunes	Desenvolvimento Web	8.50
Carla Mendes	Computação em Nuvem	6.50
Henrique Alves	Banco de Dados	6.00

Query 3 — GROUP BY: indicadores por curso

```

SELECT c.nome AS curso, COUNT(m.id) AS matriculados,
       ROUND(AVG(m.progresso), 1) AS progresso_medio,
       ROUND(AVG(m.nota), 2) AS nota_media
FROM curso c LEFT JOIN matricula m ON m.curso_id = c.id
GROUP BY c.nome
ORDER BY matriculados DESC, c.nome;

```

curso	matriculados	progresso_medio	nota_media
Computação em Nuvem	6	80.0	7.77
Banco de Dados	5	88.0	7.40
Desenvolvimento Web	5	70.0	7.55
Introdução à IA	4	38.8	9.00

Observação: AVG(m. nota) ignora notas NULL (alunos que ainda não fizeram a avaliação), por isso a média de Introdução à IA considera apenas uma nota. Isso é um comportamento padrão do SQL que precisa ser lembrado ao interpretar relatórios.

2.4 Snapshot

The screenshot shows a database management interface with three tabs at the top: "<tutordb> Script", "<tutordb> Script-1", and "<tutordb> Script-2". The active tab, "<tutordb> Script-2", contains the following SQL query:

```
SELECT ra, nome, email FROM aluno WHERE semestre = 4 ORDER BY nome;
```

Below the query editor, a table titled "aluno 1" displays the results of the query. The table has four columns: "A-Z ra", "A-Z nome", "A-Z email", and "Valor". The "Valor" column is currently selected, and its values are displayed in a separate pane on the right.

	A-Z ra	A-Z nome	A-Z email	Valor
1	2025001	Ana Souza	ana.souza@fatec.sp.gov.b	2025001
2	2025002	Bruno Lima	bruno.lima@fatec.sp.gov.	
3	2025006	Felipe Nunes	felipe.nunes@fatec.sp.gov	
4	2025010	João Pereira	joao.pereira@fatec.sp.gov	

The interface includes a sidebar on the left with icons for various database operations, a bottom toolbar with buttons like "Atualizar", "Salvar", and "Cancelar", and a "Painéis" panel on the right.

The screenshot shows a database management interface with three tabs at the top: "<tutordb> Script", "<tutordb> Script-1", and "<tutordb> Script-2". The active tab is "<tutordb> Script-2", which contains the following SQL query:

```
SELECT a.nome AS aluno, c.nome AS curso, m.nota
FROM matricula m JOIN aluno a ON a.id = m.aluno_id JOIN curso c ON c.id = m.curso_id
WHERE m.progresso = 100 ORDER BY m.nota DESC;
```

Below the query editor, there is a table view showing the results of the query. The table has three columns: "aluno(+)", "curso", and "nota". The results are as follows:

	aluno(+)	curso	nota
1	Elisa Prado	Banco de Dados	123
2	Ana Souza	Banco de Dados	
3	João Pereira	Computação em Nuvem	

At the bottom of the interface, there is a toolbar with buttons for "Atualizar", "Salvar", "Cancelar", and other navigation controls.

The screenshot shows a database management tool with a SQL editor at the top and a results pane at the bottom. The SQL script is as follows:

```
SELECT c.nome AS curso, COUNT(m.id) AS matriculados,
       ROUND(AVG(m.progresso),1) AS progresso_medio, ROUND(AVG(m.nota))
FROM curso c LEFT JOIN matricula m ON m.curso_id = c.id
GROUP BY c.nome ORDER BY matriculados DESC, c.nome;
```

The results pane displays a table with the following data:

Grade	A-Z curso	123 matriculac	123 progresso
1	Computação em N	6	80
2	Banco de Dados	5	88
3	Desenvolvimento V	5	70
4	Introdução à IA	4	38,8

O snapshot manual foi criado com o script `04_rds_snapshot.sh` (equivalente no console: Databases → selecionar a instância → Actions → Take snapshot). O RDS faz uma cópia do volume de armazenamento no S3 gerenciado pela AWS; em instâncias Single-AZ pode haver uma breve suspensão de I/O no início. O processo levou alguns minutos até o status `*available*`.

```
aws rds create-db-snapshot --db-instance-identifier fatec-skyfall-rds \
  --db-snapshot-identifier fatec-skyfall-rds-snap-AAAAAMDD-HHMM
aws rds wait db-snapshot-available --db-snapshot-identifier <id>
```

Identificador gerado: `fatec-skyfall-rds-snap-20261001-2359`

Snapshot manual × backup automático: o backup automático (retenção de 1 a 35 dias) permite restauração `*point-in-time*` e é apagado junto com a instância ou ao fim da retenção; o snapshot manual é permanente até ser excluído e é o recurso indicado antes de mudanças arriscadas (migração de schema, upgrade de versão). A restauração sempre cria uma **nova** instância a partir do snapshot.

DescribeDBSnapshots		
Criado	Id	Status
2026-10-01T23:59:30.963000+00:00	fatec-skyfall-rds-snap-20261001-2359	available

Seção 3 — Amazon DynamoDB

3.1 Modelagem da Tabela

Cenário (estado-de-atualização): cada interação do aluno com o Sistema Tutor (login, abrir aula, assistir vídeo, responder exercício, perguntar ao tutor, baixar material, logout, erro do app) gera um evento que deve ser gravado em tempo real em um log de acesso, registrando **quem** (usuário) e **quando** (timestamp) e o que aconteceu. É um fluxo de escrita contínua e de altíssimo volume, que quase nunca é atualizado — apenas inserido e consultado por aluno ou por curso.

Tabela	fatec-skyfall-eventos
Partition Key (HASH)	usuariold (String) — ex.: aluno-2025001
Sort Key (RANGE)	dataHoraEvento (String) — ex.: 2026-09-28T08:01:12Z#evt-0001
GSI	gsi-curso-data — PK cursold, SK dataHoraEvento, projeção ALL
Modo de capacidade	On-demand (PAY_PER_REQUEST)
TTL	Atributo expiraEm (epoch) — eventos expiram automaticamente

Por que usuariold como Partition Key? A pergunta mais comum do sistema é "o que este aluno fez?" (linha do tempo, progresso, recomendações do tutor), e a Partition Key é a única forma de ler dados sem varrer a tabela. Além disso, há muitos usuários distintos (alta cardinalidade), o que espalha as escritas entre muitas partições físicas — essencial para alto volume. Uma PK como tipoEvento ou a data do dia concentraria todas as escritas em poucas partições (*hot partition*) e causaria throttling.

Por que dataHoraEvento como Sort Key? Datas em ISO-8601 (UTC) ordenam lexicograficamente na mesma ordem que cronologicamente, então o DynamoDB devolve os eventos de um aluno já ordenados e permite filtros de intervalo (BETWEEN, begins_with) direto na chave. O sufixo #eventoId garante unicidade da chave primária quando dois eventos do mesmo aluno acontecem no mesmo milissegundo — sem ele, o segundo PutItem sobrescreveria o primeiro.

On-demand e TTL: eventos de usuários têm picos (início de aula, véspera de prova); o modo on-demand escala automaticamente sem planejar capacidade. O TTL apaga eventos antigos sem custo de escrita, mantendo a tabela enxuta.

Padrão de acesso	Como é atendido
Linha do tempo de um aluno	Query PK usuariold
Eventos de um aluno em um período	Query PK usuariold + SK BETWEEN data1 AND data2
Último evento do aluno ("estado atual")	Query PK + ScanIndexForward=false + Limit 1
Atividade de um curso (painel do professor)	Query no GSI gsi-curso-data (PK cursold)
Exercícios respondidos em um curso	Query no GSI + FilterExpression tipoEvento

Informações gerais

[Informações](#)

Chave de partição
usuarioid (String)

Alarmes
🟢 Nenhum alarme ativo

Tamanho médio do item
0 bytes

Nome do recurso da Amazon (ARN)
arn:aws:dynamodb:us-east-1:161800939960:table/fatec-skyfall-eventos

Chave de classificação
dataHoraEvento (String)

Recuperação em um ponto anterior no tempo (PITR) [Informações](#)
🕒 Desativado

Política baseada em recursos [Informações](#)
🕒 Não ativo

Modo de capacidade
[Sob demanda](#)

Contagem de itens
0

Status da tabela
🟢 Ativo

Tamanho da tabela
0 bytes

[Obter contagem de itens ativos](#)

► **Informações adicionais**

Índices secundários globais (1)

[Informações](#)

[Excluir](#) [Criar índice](#)

< 1 > ⚙️

	Nome	Status	Chave de partição	Chave de classificação	Capacidade de leitura	Capacidade de g
☐	gsi-curso-data	🟢 Ativo	cursoId (String)	dataHoraEvento (String)	Sob demanda	Sob demanda

3.2 Dados Inseridos

Foram inseridos 18 eventos via batch-write-item (arquivo dynamodb/itens.json) e mais 1 via put-item, totalizando **19 itens** de 6 usuários e 9 tipos de evento. Todos os itens têm em comum apenas usuarioId, dataHoraEvento, eventoId, tipoEvento e expiraEm; os demais atributos variam conforme o tipo de evento, explorando o schema flexível do DynamoDB — inclusive tipos String, Number, Boolean, List e Map:

usuarioid	dataHoraEvento (início)	tipoEvento	Atributos específicos
aluno-2025001	2026-09-28T08:01:12Z	LOGIN	dispositivo, ip, navegador
aluno-2025001	2026-09-28T08:03:40Z	ACESSO_AULA	cursoId, aulaId, dispositivo
aluno-2025001	2026-09-28T08:25:02Z	EXERCICIO_RESPONDIDO	cursoId, exercicioId, acertou (BOOL), tentativas, tempoSeg
aluno-2025001	2026-09-29T19:10:55Z	PERGUNTA_TUTOR	cursoId, pergunta, tokens
aluno-2025001	2026-09-29T19:40:00Z	LOGOUT	dispositivo, duracaoSessaoSeg
aluno-2025002	2026-09-28T10:00:00Z	LOGIN	dispositivo, versaoApp
aluno-2025002	2026-09-28T10:02:31Z	VIDEO_ASSISTIDO	cursoId, aulaId, percentual, velocidade
aluno-2025002	2026-09-30T21:15:09Z	DOWNLOAD_MATERIAL	cursoId, arquivoS3, tamanhoKB
aluno-2025003	2026-09-29T14:20:00Z	ACESSO_AULA	cursoId, aulaId, dispositivo
aluno-2025003	2026-09-29T14:48:13Z	EXERCICIO_RESPONDIDO	cursoId, exercicioId, acertou, tentativas, dicasUsadas (LIST)
aluno-2025003	2026-10-01T09:05:44Z	CONQUISTA_DESBLOQUEADA	cursoId, medalha, pontos
aluno-2025005	2026-09-30T07:30:00Z	LOGIN	dispositivo, mfa (BOOL)
aluno-2025005	2026-09-30T07:31:18Z	ACESSO_AULA	cursoId, aulaId, dispositivo
aluno-2025005	2026-09-30T07:59:47Z	EXERCICIO_RESPONDIDO	cursoId, exercicioId, acertou, tentativas, tempoSeg

usuarioId	dataHoraEvento (início)	tipoEvento	Atributos específicos
aluno-2025005	2026-10-01T18:12:03Z	PERGUNTA_TUTOR	cursorId, pergunta, tokens, avaliacaoResposta
aluno-2025008	2026-09-28T23:55:21Z	ERRO_APP	dispositivo, codigoErro, detalhe (MAP)
aluno-2025008	2026-09-29T00:02:10Z	ACESSO_AULA	cursorId, aulaId, dispositivo
aluno-2025010	2026-10-01T13:45:00Z	VIDEO_ASSISTIDO	cursorId, aulaId, percentual, velocidade
aluno-2025010	(hora da execução)	LOGIN	dispositivo, executadoPor — inserido via put-item

Os `usuarioId` correspondem aos RAs dos alunos cadastrados no RDS, o que permite à API cruzar o log de eventos com os dados acadêmicos.

✔ Concluído Itens devolvidos: 19 Itens verificados: 19 Eficiência: 100% RCUs consumidos: 2

Tabela: fatec-skyfall-eventos - Itens retornados (19)

Ações

Criar item

Verificação iniciada em outubro 01, 2026, 21:18:15

<

1

>

☐	usuarioid (String)	dataHoraEvento (String)	acertou	arquivoS3	aulaid	avaliacaoResposta
☐	aluno-2025005	2026-09-30T07:30:00Z#evt-...				
☐	aluno-2025005	2026-09-30T07:31:18Z#evt-...			BD-01-A06	
☐	aluno-2025005	2026-09-30T07:59:47Z#evt-...	true			
☐	aluno-2025005	2026-10-01T18:12:03Z#evt-...				5
☐	aluno-2025002	2026-09-28T10:00:00Z#evt-...				
☐	aluno-2025002	2026-09-28T10:02:31Z#evt-...			WEB-01-A02	
☐	aluno-2025002	2026-09-30T21:15:09Z#evt-...		publico/gui...		

3.3 Queries e GSI

Todas as consultas abaixo usam Query (leitura direta pela chave), nunca Scan. As saídas completas ficam em `evidencias/dynamodb.log`.

#	Consulta	Chave usada	Resultado esperado
Q1	Linha do tempo de aluno-2025001	PK	5 eventos, em ordem cronológica
Q2	Eventos de aluno-2025005 em 30/09/2026	PK + SK BETWEEN	3 eventos (LOGIN, ACESSO_AULA, EXERCICIO_RESPONDIDO)
Q3	Último evento de aluno-2025003	PK, ordem decrescente, Limit 1	CONQUISTA_DESBLOQUEADA de 01/10/2026
Q4	Todos os eventos do curso BD-01	GSI (cursorId)	6 eventos de 3 alunos diferentes
Q5	Exercícios respondidos em NUV-01	GSI + FilterExpression	1 evento (aluno-2025003, acertou = false)

```
aws dynamodb query --table-name fatec-skyfall-eventos \
--key-condition-expression "usuarioId = :u AND dataHoraEvento BETWEEN :ini AND :fim" \
--expression-attribute-values '{"u":{"S":"aluno-2025005"},
":ini":{"S":"2026-09-30T00:00:00Z"},"fim":{"S":"2026-09-30T23:59:59Z~"}}'
```

```
aws dynamodb query --table-name fatec-skyfall-eventos --index-name gsi-curso-data \
--key-condition-expression "cursoId = :c" \
--expression-attribute-values '{"c":{"S":"BD-01"}}'
```

O que o GSI permite: a tabela principal só sabe responder perguntas "por aluno". Sem o GSI, saber o que aconteceu no curso BD-01 exigiria um Scan — ler a tabela inteira e descartar o que não interessa —, cujo custo e tempo crescem com o tamanho total da tabela (com milhões de eventos, inviável). O GSI mantém automaticamente uma segunda organização dos mesmos dados com cursoId como Partition Key, e a consulta passa a ler apenas os itens daquele curso, já ordenados por data. É um **índice esparsos**: eventos sem cursoId (LOGIN, LOGOUT, ERRO_APP) simplesmente não entram nele, o que o deixa menor e mais barato. Contrapartida: o GSI tem consistência eventual e cada escrita na tabela gera uma escrita extra no índice (custo).

```
atv3 $ aws dynamodb query --table-name $TABELA --index-name gsi-curso-data \
> --key-condition-expression "cursoId = :c" --expression-attribute-values '{"c":{"S":"BD-01"}}' \
> --projection-expression "cursoId, dataHoraEvento, usuarioId, tipoEvento" --output table
```

Query		
ConsumedCapacity	Count	ScannedCount
None	6	6

Items	
cursoId	
S	BD-01

dataHoraEvento	
S	2026-09-28T08:03:40Z#evt-0002

tipoEvento	
S	ACESSO_AULA

usuarioId	
S	aluno-2025001

Items	
cursoId	
S	BD-01

3.4 Operação via CLI

Toda a Seção 3 foi executada via AWS CLI pelo script v1-cli/05_dynamodb.sh (criação da tabela e do GSI, TTL, carga em lote, put-item, cinco queries e contagem). Cada comando é gravado com o nome do executor e a data/hora no arquivo evidencias/dynamodb.log. Exemplo de inserção individual:

```
aws dynamodb put-item --table-name fatec-skyfall-eventos --item '{
  "usuarioId": {"S": "aluno-2025010"},
  "dataHoraEvento": {"S": "<agora>#evt-cli-01"},
  "eventoId": {"S": "evt-cli-01"},
  "tipoEvento": {"S": "LOGIN"},
  "dispositivo": {"S": "cli"},
  "executadoPor": {"S": "<nome do integrante>"}
}' --return-consumed-capacity TOTAL
```

Executado por: **Eduardo Kuwahara 01/10/2026 às 21h09** . Saída retornada (conferir com evidencias/dynamodb.log):

```
{
  "ConsumedCapacity": {
    "TableName": "fatec-skyfall-eventos",
```

```

    "CapacityUnits": 1.0
  }
}

```

Nota: como este evento LOGIN não tem cursoId, ele não é gravado no GSI; o consumo esperado é de 1 WCU (item < 1 KB). Um evento com cursoId consumiria também a escrita no índice.

```

atv3 $ aws dynamodb query --table-name $TABELA --key-condition-expression "usuarioId = :u" \
> --expression-attribute-values '{":u":{"S":"aluno-2025001"}}' \
> --projection-expression "dataHoraEvento, tipoEvento" --output table
-----
|                                     Query                                     |
+-----+-----+-----+
| ConsumedCapacity | Count | ScannedCount |
+-----+-----+-----+
| None            | 5     | 5            |
+-----+-----+-----+
|                                     Items                                     |
| dataHoraEvento  |
+-----+-----+-----+
| S | 2026-09-28T08:01:12Z#evt-0001 |
+-----+-----+-----+
| tipoEvento      |
+-----+-----+-----+
| S | LOGIN |
+-----+-----+-----+
|                                     Items                                     |
| dataHoraEvento  |
+-----+-----+-----+
| S | 2026-09-28T08:03:40Z#evt-0002 |
+-----+-----+-----+
| tipoEvento      |
+-----+-----+-----+
| S | ACESSO_AULA |
+-----+-----+-----+
|                                     Items                                     |
| dataHoraEvento  |
+-----+-----+-----+
| S | 2026-09-28T08:25:02Z#evt-0003 |
+-----+-----+-----+

```

Executado por Fábio Hiroshi às 01/10/2026 às 21h09

3.5 Teste de alto volume de escrita (requisito não-funcional)

Para comprovar o requisito de "altíssimo volume de escrita" de forma mensurável, o grupo definiu a métrica:

RNF-01 — O registro de eventos deve sustentar no mínimo 1.000 eventos/s, sem perda de eventos (0 falhas) e com latência p95 por lote de no máximo 500 ms, medida no cliente (incluindo o trajeto pela internet).

Método: o script `v1-cli/06_dynamodb_carga.py` (Python + boto3) gera eventos sintéticos para 5.000 usuários diferentes e os envia com `BatchWriteItem` (25 itens por chamada) em 16 threads paralelas. Itens não processados por throttling são reenviados com `*backoff*` exponencial e contabilizados. O script mede vazão por segundo e latência de cada lote, e imprime o resultado em JSON no snap-20261001-2359. Os itens de carga têm TTL de 1 dia, então desaparecem sozinhos. Do lado da AWS, a confirmação vem da métrica `ConsumedWriteCapacityUnits` do CloudWatch.

Métrica	Meta (RNF-01)	Resultado obtido
Eventos enviados / gravados	—	20.000
Eventos perdidos	0	0
Duração do teste	—	4.44

Métrica	Meta (RNF-01)	Resultado obtido
Vazão média	≥ 1.000 eventos/s	4502,2
Latência por lote p50 / p95 / p99	$p95 \leq 500$ ms	43.8/171.7/231.6
Itens reenviados por throttling	—	411
RNF-01 atendido?	Sim	Sim

Resultados copiados da saída do script `carga.py`. Executado por **Paulo Almeida** a partir de **AWS CloudShell**.

```

ativ3 $ python3 carga.py
[
  "eventos_gravados": 20000,
  "eventos_perdidos": 0,
  "reenviados_throttling": 411,
  "duracao_s": 4.44,
  "vazao_eventos_por_s": 4504.2,
  "latencia_lote_ms": {
    "p50": 43.8,
    "p95": 171.7,
    "p99": 231.6
  },
  "RNF01_atendido": true
]

```



Análise: A tabela gravou 20.000 eventos em 4,44 s, a cerca de 4.504 eventos/s, sem nenhuma perda e com latência p95 de 171,7 ms, superando as metas do RNF-01 (1.000 eventos/s e p95 ≤ 500 ms). Houve 411 itens reenviados por throttling, todos recuperados pelo backoff exponencial, o que é esperado em uma tabela on-demand nova submetida a um pico repentino. Como o teste partiu do CloudShell, dentro da AWS, a rede teve pouco peso na latência. Uma tabela on-demand nova aceita imediatamente picos de alguns milhares de escritas por segundo e se adapta automaticamente a picos maiores; por isso, em testes a partir de um único computador, o gargalo costuma ser a rede e o número de conexões do cliente, e não o banco — exatamente o comportamento desejado para um log de eventos.

3.6 Comparação DynamoDB x SQL

A mesma tabela de eventos, representada em SQL (PostgreSQL):

```
CREATE TABLE evento (
  usuario_id      VARCHAR(30) NOT NULL,
  data_hora_evento TIMESTAMPTZ NOT NULL,
  evento_id       VARCHAR(20) NOT NULL,
  tipo_evento     VARCHAR(30) NOT NULL,
  curso_id        VARCHAR(10) REFERENCES curso(codigo), -- opcional
  expira_em       TIMESTAMPTZ,
  dados           JSONB, -- atributos variáveis (aulaId, nota, dispositivo...)
  PRIMARY KEY (usuario_id, data_hora_evento, evento_id)
);
CREATE INDEX idx_evento_curso_data ON evento (curso_id, data_hora_evento); -- equivale ao GSI
-- TTL não existe nativamente: seria um job agendado
-- DELETE FROM evento WHERE expira_em < now();
```

Aspecto	DynamoDB (NoSQL)	SQL (RDS PostgreSQL)
Schema	Só as chaves são fixas; cada item tem os atributos que quiser. Novos tipos de evento não exigem migração.	Colunas fixas e tipadas; atributos variáveis precisam de colunas NULL ou de um campo JSONB. Mudanças exigem ALTER TABLE.
Joins	Não existem. Os dados são desnormalizados e modelados a partir dos padrões de acesso (ex.: cursold copiado no evento).	JOIN nativo entre evento, aluno e curso; o modelo nasce normalizado e as consultas são livres.
Consultas	Eficientes apenas por chave (PK, PK+SK, GSI). Consultas ad hoc exigem Scan.	Qualquer combinação de filtros, agregações (GROUP BY) e subconsultas.

Aspecto	DynamoDB (NoSQL)	SQL (RDS PostgreSQL)
Escalabilidade	Horizontal e automática (particionamento por PK); milhares de escritas/s sem administrar servidor.	Principalmente vertical (instância maior); escrita concentrada em um único nó primário; réplicas só para leitura.
Consistência	Eventual por padrão; leitura fortemente consistente opcional na tabela (não no GSI); transações limitadas.	Forte (ACID) em todas as operações, com transações envolvendo várias tabelas.
Expiração (TTL)	Nativa e sem custo de escrita.	Precisa de job agendado (cron/pg_cron) com DELETE.
Custo	Por requisição (on-demand) ou capacidade provisionada; custo zero ocioso, exceto armazenamento.	Por hora de instância, mesmo sem uso.

Conclusão do grupo: para o log de eventos, o DynamoDB é mais adequado porque os padrões de acesso são conhecidos e simples (por aluno e por curso, ordenados por tempo) e o volume de escrita é alto e imprevisível. Já os dados acadêmicos (alunos, matrículas, notas), que exigem integridade e relatórios variados, continuam no SQL.

Seção 4 — Análise Comparativa

4.1 Custo estimado: qual cenário seria mais caro?

Estimativa mensal em us-east-1, fora do free tier, para o porte deste projeto (valores de referência aproximados; preços exatos devem ser conferidos na AWS Pricing Calculator):

Serviço	Premissa de uso	Custo aproximado/mês
S3	1 GB armazenado + ~10 mil requisições GET	≈ US\$ 0,03
RDS PostgreSQL	db.t3.micro ligado 24x7 (≈ US\$ 0,018/h × 730 h) + 20 GB gp2	≈ US\$ 15
DynamoDB on-demand	1 milhão de escritas + 1 milhão de leituras + 1 GB	≈ US\$ 1

O RDS é o mais caro neste cenário, porque é cobrado por hora de instância, esteja o banco sendo usado ou não — às 3 da manhã, sem nenhum aluno, ele custa o mesmo. Multi-AZ ou uma instância maior multiplicariam esse valor. S3 e DynamoDB on-demand são **serverless**: pagam por GB e por requisição, então em baixa escala custam centavos. A relação pode se inverter em escala muito alta e constante: 1.000 escritas/s ininterruptas somam cerca de 2,6 bilhões de escritas por mês, o que no DynamoDB on-demand chega a milhares de dólares — nesse caso migra-se para capacidade provisionada (com auto scaling e reservas), que reduz bastante o custo. Ou seja, o custo depende do perfil de uso: RDS tem custo fixo previsível; DynamoDB tem custo variável proporcional ao tráfego.

4.2 Controle de schema: vantagem ou desvantagem?

O RDS oferece o maior controle sobre o schema: tipos de dados, NOT NULL, UNIQUE, CHECK (nota entre 0 e 10, semestre entre 1 e 6) e chaves estrangeiras são garantidos pelo próprio banco, independentemente de bugs da aplicação. No DynamoDB, só as chaves são obrigatórias — a validação fica toda no código da API —, e o S3 não tem schema algum (apenas objetos e metadados). Para o

grupo, esse controle é **vantagem** quando os dados são o "registro oficial" e se relacionam entre si: uma matrícula nunca pode apontar para um aluno inexistente, e uma nota inválida é rejeitada na hora. É **desvantagem** quando os dados mudam de forma com frequência ou são heterogêneos, como os eventos: cada novo tipo de evento exigiria ALTER TABLE e migração, enquanto no DynamoDB basta gravar o novo atributo. Por isso a resposta não é única: usamos schema rígido onde a integridade é o requisito principal e schema flexível onde a evolução rápida e o volume são o requisito principal.

4.3 Backup gerenciado: qual serviço entrega isso mais facilmente?

O **Amazon RDS** entrega o backup gerenciado mais completo com menos esforço: os backups automáticos já vêm ligados na criação da instância (retenção configurável de 1 a 35 dias), com logs de transação que permitem restauração **point-in-time** para qualquer segundo dentro da janela, além de snapshots manuais com um clique (como fizemos na seção 2.4) e cópia de snapshots para outra região. O DynamoDB fica muito próximo: o **Point-in-Time Recovery** (35 dias) é ativado com um único comando, e há backups sob demanda sem impacto no desempenho — mas o PITR vem desligado por padrão. O S3 não tem "backup" no mesmo sentido: a proteção vem do versionamento (que usamos), da replicação entre regiões e da alta durabilidade (11 nozes). Para uma empresa que quer "backup automático sem pensar nisso", o RDS é a resposta mais direta; o AWS Backup pode centralizar as políticas dos três serviços em um único lugar.

4.4 App mobile com 1 milhão de usuários: qual serviço vocês recomendam?

Recomendamos o **DynamoDB como banco principal das funcionalidades de alto tráfego**, combinado com **S3 (+ CloudFront) para arquivos**. Fazendo uma conta simples: se 10% dos usuários forem ativos por dia (100 mil) e cada um gerar 50 eventos, são 5 milhões de escritas por dia — cerca de 60/s em média, mas com picos facilmente 10 a 20 vezes maiores em horários de aula ou véspera de prova. O DynamoDB absorve esses picos sem intervenção, com latência de milissegundos de um dígito em qualquer volume, sem limite de conexões (cada requisição é HTTP, ideal para milhares de dispositivos e funções serverless) e com replicação multi-região (Global Tables) se o app crescer para fora do país. Um RDS na mesma situação exigiria dimensionar a instância para o pico, gerenciar o limite de conexões (RDS Proxy), réplicas de leitura e janelas de manutenção.

Isso não elimina o banco relacional: dados financeiros, matrículas e relatórios acadêmicos, que exigem transações ACID e consultas variadas, continuariam em RDS (ou Aurora), que tem volume muito menor. Imagens, vídeos e PDFs ficariam no S3 entregues via CDN, e o app faria upload direto ao S3 por URL pré-assinada, sem sobrecarregar a API — exatamente a arquitetura implementada na Versão 2 deste trabalho.

Seção 5 — Versão 2: armazenamento via API

Na Versão 2, os mesmos recursos passam a ser usados **programaticamente** pela API do Sistema Tutor (pasta `v2-api/`, FastAPI + AWS SDK boto3 + psycpg), aplicando a decisão de armazenamento descrita no início do relatório. Cada endpoint corresponde a uma possibilidade de decisão:

Endpoint	Serviço	O que demonstra
GET /saude	S3, RDS, DynamoDB	Verifica a conectividade com os três serviços

Endpoint	Serviço	O que demonstra
GET /materiais	S3	Lista objetos (ListObjectsV2) e monta URL pública dos arquivos de publico/
POST /materiais/upload-url	S3	Gera URL pré-assinada de PUT: o cliente envia o arquivo direto ao S3, sem passar pela API
GET /materiais/download-url	S3	Acesso temporário (expira) a arquivo privado — ex.: privado/alunos.csv
GET /materiais/versoes	S3	Consulta as versões de um objeto (ListObjectVersions)
GET /alunos?semestre=4	RDS	SELECT com filtro (Query 1)
GET /alunos/{ra}/boletim	RDS	JOIN aluno × matrícula × curso
GET /cursos/indicadores	RDS	GROUP BY (Query 3)
POST /eventos	DynamoDB	PutItem de um evento em tempo real (com TTL e índice esparso)
GET /eventos/aluno/{id}	DynamoDB	Query por Partition Key + intervalo na Sort Key
GET /eventos/curso/{id}	DynamoDB	Query no GSI gsi-curso-data, com filtro opcional por tipo

Execução: `pip install -r requirements.txt` e `uvicorn app:app --reload`; a documentação interativa fica em <http://127.0.0.1:8000/docs>. A API reutiliza o `config.env` da Versão 1 e descobre o endpoint do RDS pela própria API da AWS (`DescribeDBInstances`). Nenhuma credencial fica no código: o SDK usa o perfil configurado no AWS CLI (ou a role IAM, se publicada em EC2/Lambda).

Integração com o projeto: os endpoints de materiais alimentam a tela de conteúdos das trilhas; os de RDS atendem o painel acadêmico (boletim e indicadores por curso); e o `POST /eventos` é chamado pelo front a cada interação do aluno, alimentando o histórico usado pelo tutor para recomendar conteúdos. Esses módulos serão incorporados à API do grupo conforme o planejamento das sprints.

Seção 6 — Reflexão do Grupo

O primeiro ponto que exigiu estudo foi o modelo de permissões do S3. Buckets novos vêm com ACLs desabilitadas e com o Block Public Access ligado, então não basta marcar um arquivo como público: foi preciso entender a diferença entre ACL e bucket policy. A solução final, liberando apenas o prefixo `publico/` por política, ficou mais organizada e mais segura do que liberar arquivos um a um, e separa claramente o que é material aberto do que é dado pessoal protegido pela LGPD.

No RDS, a principal lição foi sobre rede e segurança: a instância só aceita conexões do IP liberado no Security Group, o que protege o banco, mas exige atualizar a regra sempre que o integrante muda de rede. Também chamou a atenção do grupo o tempo de provisionamento da instância e o modelo de cobrança por hora, que vale mesmo com o banco ocioso — por isso foi criado um script de limpeza para remover os recursos ao final.

O DynamoDB foi o serviço que mais exigiu mudança de mentalidade. Acostumados a pensar em tabelas normalizadas e JOINS, tivemos que começar a modelagem pelas perguntas que o sistema precisa responder. A discussão sobre qual atributo usar como Partition Key, e o entendimento do problema de partição quente (hot partition), foi o principal aprendizado técnico da atividade, assim como a percepção de que um GSI permite responder a uma nova pergunta sem recorrer a Scan.

Foi mais simples do que esperávamos automatizar tudo com a AWS CLI. Com os scripts versionados no repositório, o ambiente pode ser recriado do zero e cada comando fica registrado com sua saída, o que serviu como evidência e facilitou a montagem deste relatório. O teste de carga também mostrou que, a partir de um único computador, o limite tende a estar no cliente e na rede, e não no serviço da AWS. Por fim, a atividade deixou claro que não existe um armazenamento melhor em absoluto: a escolha depende do tipo de dado e do padrão de acesso, e por isso a API do grupo combina os três serviços.

Anexo A — Mapa de evidências (rubrica)

Critério da rubrica	Onde está no relatório	Evidência no repositório
Implementação S3 (upload, permissões, versionamento, regras)	Seção 1	v1-cli/01_s3.sh · evidencias/s3.log · s3_bucket_policy.json · s3_lifecycle.json
Implementação RDS (instância, conexão, queries, snapshot)	Seção 2	v1-cli/02, 03, 04 · sql/ · evidencias/rds*.log
Implementação DynamoDB (modelagem, GSI, CLI, comparação SQL)	Seção 3	v1-cli/05_dynamodb.sh · 06_dynamodb_carga.py · evidencias/dynamodb*.{log,json,png}
Análise comparativa	Seção 4	—
Qualidade das entregas (prints, outputs, membro)	Prints em todas as seções; logs com nome do executor	evidencias/
Versão 2 — uso via API	Seção 5	v2-api/app.py

Referências

- AWS. Amazon S3 — Guia do usuário. https://docs.aws.amazon.com/pt_br/AmazonS3/latest/userguide/
- AWS. Bloqueio do acesso público e políticas de bucket. https://docs.aws.amazon.com/pt_br/AmazonS3/latest/userguide/access-control-block-public-access.html
- AWS. Gerenciamento do ciclo de vida de objetos no S3. https://docs.aws.amazon.com/pt_br/AmazonS3/latest/userguide/object-lifecycle-mgmt.html
- AWS. Amazon RDS — Guia do usuário. https://docs.aws.amazon.com/pt_br/AmazonRDS/latest/UserGuide/
- AWS. Amazon DynamoDB — Guia do desenvolvedor (boas práticas de chaves de partição). https://docs.aws.amazon.com/pt_br/amazondynamodb/latest/developerguide/best-practices.html
- AWS. AWS CLI Command Reference. <https://docs.aws.amazon.com/cli/latest/>
- AWS. Boto3 Documentation. <https://boto3.amazonaws.com/v1/documentation/api/latest/index.html>
- AWS. AWS Pricing Calculator. <https://calculator.aws/>